

CODE: CTC-153-26

writer: SIRAJUDIN SEID

vulnerable web; <https://labs.tibeb.org/5bf29de8-1e83-4b9e-9ee9-2636b9387055/>

date Aug-05-2026

vulnerabilities found

- sql injection
- XSS vulnerability
- race condition
- IDOR
- Business Logic Flaw combined with Improper Input Validation

Result 1 SQL injection

Summary

A SQL Injection vulnerability was identified in the VulnBank web app. The application fails to properly sanitize or parameterize user input within the login fields before passing it to the database engine. This allows an unauthenticated attacker to inject malicious SQL commands, bypass the authentication mechanism, and gain unauthorized administrative access to the application

Severity is “Critical”

Reproduction Steps

- 1 Navigate to the VulnBank web application.
- 2 Enter a malicious SQL payload into the username field ' OR 1=1 --
- 3 Enter any arbitrary value into the password field.
- 4 Click the "Login" button to submit the form.
- 5 Observe that the authentication mechanism is bypassed and the application

PoC



VulnBank

Login

' OR 1=1 --

•••••

Login

[Don't have an account? Register](#)

Welcome back, ADMIN

 **Transfer Money**

To Account ID

Amount

Send Money

 **My Comments**

Post Comment

 **All Customers**

ID	Username	Email	Balance
1	ADMIN	admin@bank.com	\$81294912926.00
2	user1	user1@bank.com	\$550.00
3	test	bee@bee.bee	\$-999099.00
4	IDFgwqOx	james@gmail.com	\$1100.00
5	one	one@gmail.com	\$1000.00
6	wiener	keb@gmail.com	\$1000.00
7		d@DA DA	\$1000.00

Impact

An attacker can exploit this vulnerability to bypass authentication entirely, granting them full administrative control over the application backend. This compromises the entire database, resulting in a total loss of confidentiality, integrity, and availability. The attacker can read sensitive user data, modify or delete critical records, extract intellectual property, or leverage admin privileges to execute remote code on the underlying host server

Remediation

implement a Prepared statements separate the code from the data. The database compiles the SQL query structure first, then inserts the user input strictly as a text value. Even if an attacker enters SQL commands, the database treats it as a harmless username string

Result 2 XSS injection

Summary

A Stored Cross-Site Scripting (XSS) vulnerability was identified in the "My Comments" feature of the authenticated user dashboard at the endpoint `/5bf29de8-1e83-4b9e-9ee9-2636b9387055/comment`. The web application allows users to input text via the comment submission box and stores this data directly in the database without performing proper context-aware sanitization or output encoding. When the comment history is rendered back to the user, an attacker can break out of existing HTML constraints (such as a `<textarea>` tag) and force the

victim's browser to execute arbitrary JavaScript code within the context of their session

Severity is “High”

Reproduction Steps

1. Log in to the application and navigate to the main dashboard panel showing account balances and administrative features.
2. In the "My Comments" submission box, input an XSS payload designed to break out of text containers and execute JavaScript
</textarea><script>alert("stored XSS")</script>).
3. Click the "Post Comment" button to submit and save the text payload to the server database.
4. Refresh the page or navigate away and return to the dashboard view.
5. Observe that the browser parses the stored input as executable code, firing a popup window displaying the string "stored XSS", proving successful arbitrary execution.

PoC

\$-10910091897790008.00

Welcome back, admin

Transfer Money

To Account ID

Enter recipient ID

Amount

Amount

My Comments

```
</textarea><script>alert("stored XSS")(</script>
```

Post Comment

Comment posted successfully!

Available Balance

\$0.00

Welcome back, User

Transfer Money

To Account ID

Enter recipient ID

Amount

Amount

Send Money

My Comments

Write a comment...

Post Comment

user
Comment text

Available Balance

\$-999999999995355401293922304.00

Welcome back, admin

Transfer Money

To Account ID

Enter recipient ID

Amount

Amount

labs.tibeb.org

stored XSS

☐ Don't allow labs.tibeb.org to prompt you again

OK

Impact

An attacker can exploit this vulnerability to execute malicious JavaScript silently inside the session of any user who loads their dashboard or comment history. Because the dashboard contains sensitive functions like "Transfer Money" and financial balances, a malicious script can automatically forge unauthorized actions (Cross-Site Request Forgery) or access local session data. If session tokens lack defensive attributes like the HttpOnly cookie flag, an attacker can leverage this stored flaw to exfiltrate session data to a remote server, leading to administrative account hijacking and total session compromise.

Remediation

=> before echoing any user input back into the HTML, you should encode any potentially dangerous characters as their corresponding HTML entities (so that < would become < and > would become >), thus preventing browsers from interpreting malicious input as HTML tags or scripts.

=> make sure that the cookies which store authentication information are marked as HttpOnly, thus preventing any malicious script from reading document.cookie and stealing your authentication tokens (see this section for details).

Result 3 Race Condition Vulnerability

Summary

A Race Condition vulnerability was identified in the transfer endpoint at `/e5fa6061-07aa-49c6-9f13-13070845e4729/transfer`. The endpoint processes fund transfer requests without implementing proper transactional locking or synchronization mechanisms. When multiple transfer requests are sent concurrently (in parallel), the server checks the account balance for all requests simultaneously before any single transaction updates the balance, allowing an attacker to successfully execute multiple transfers that exceed their actual available funds.

Severity “Critical”

Remediation Steps

- 1 Log in to the application and capture a legitimate money transfer request using an interception proxy {burpsuite}
- 2 Note the parameters in the request body containing the source account, destination account, and the transfer amount
from_account=102&to_account=103&amount=1999999999994343434343439999.
- 3 Send this request to a parallel execution in burp repeater
- 4 Duplicate the request multiple times within the parallel group
- 5 Execute the group simultaneously to send all requests to the server at the exact same millisecond.
- 6 Observe that all concurrent requests return an HTTP/2 302 Found redirection status, indicating that every individual transfer succeeded despite exceeding the account balance limits.

Poc

1 2 3 money 21 < 4 5 6 7 8 9 10 + 🔍 ^

11 12 13 14 15 16 17 18 19 20 21

22 23 24 ×

Send group (parallel) ⚙ Cancel < > Follow redirection ⚡ Burp A HT

Request

Pretty Raw Hex 🔍 📄 ln ≡

```
1 POST /e5fa6061-07aa-49c6-9f13-1370845e4729/transfer
2 HTTP/2
3 Host: labs.tibeb.org
4 Cookie: vulnbank_session=
  eyJ1c2VyX2lkIjozLCJlc2VybmFtZSI6IlZpYnJvdXoifQ.anML
  Jw.vWFAAdMNeMwR1SDwXNhk9oiy8o8
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64;
  rv:140.0) Gecko/20100101 Firefox/140.0
6 Accept:
  text/html,application/xhtml+xml,application/xml;q=0
  .9,*/*;q=0.8
7 Accept-Language: en-US,en;q=0.5
8 Accept-Encoding: gzip, deflate, br
9 Content-Type: application/x-www-form-urlencoded
10 Content-Length: 66
11 Origin: null
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19 from_account=102&to_account=103&amount=
  199999999994343434343439999
```

Response

Pretty Raw Hex <Re... 🔍 📄 ln ≡

```
1 HTTP/2 302 Found
2 Date: Wed, 05 Aug 2026 12:14:18 GMT
3 Content-Type: text/html; charset=utf-8
4 Cache-Control: no-store
5 Location:
  /e5fa6061-07aa-49c6-9f13-1370845e4729/dashboard
6 Referrer-Policy: no-referrer
7 Server: cloudflare
8 X-Content-Type-Options: nosniff
9 X-Robots-Tag: noindex, nofollow, noarchive
10 Nel:
  {"report_to":"cf-nel","success_fraction":0.0,"max_a
  ge":604800}
11 Cf-Cache-Status: DYNAMIC
12 Report-To:
  {"group":"cf-nel","max_age":604800,"endpoints":[{"u
  rl":"https://a.nel.cloudflare.com/report/v4?s=3ovjd
  84Gcc%2FEUAFZxu0RMvvyGHMcecIGgBNqJc8lyjvch0%2FyCyQDw
  8YqdSzBUDLBzm3MNncIltGOeSN8uzVLPZIZSFGiE%2FwHQRsy2B
  ZQ3iiYj7QYQRN2cQB922%2FMQXfIlwg%3D%3D"}]}
13 Server-Timing: cfCacheStatus;desc="DYNAMIC"
14 Server-Timing: cfEdge;dur=4,cfOrigin;dur=579
15 Cf-Ray: a265b8810926ef9a-MRS
16 Alt-Svc: h3=":443"; ma=86400
17
18 <!doctype html>
19 <html lang=en>
20 <title>
  Redirecting...
```

Impact

An attacker can exploit this vulnerability to bypass server-side balance restrictions and multiply their funds or drain the bank's liquidity. By executing concurrent requests, the system fails to accurately subtract the transferred amounts in real time, allowing the attacker to transfer the same balance multiple times.

This leads to severe financial logic corruption, direct theft of platform capital, and an unauthorized negative/overflowed balance state that compromises the integrity of the entire ledger system

Remediation

1 Database Row Locking (FOR UPDATE) Lock the sender's account row during the initial query to force parallel requests into a sequential queue

2 Atomic Transactions (BEGIN/COMMIT): Group the balance check, deduction, and recipient credit into a single transaction that succeeds or fails as one unit.

3 Database Constraints (CHECK): Configure a schema rule on the balance column (e.g., CHECK (balance >= 0)) to force the database to automatically reject transfers that cause a negative balance

Result 4 IDOR Vulnerability

Summary

An Insecure Direct Object Reference (IDOR) vulnerability was identified in the transfer endpoint at

/5bf29de8-1e83-4b9e-9ee9-2636b9387055/transfer.

The endpoint accepts reference identifiers for both the source account (from_account) and destination account (to_account) inside the request body without verifying if the authenticated session possesses ownership or authorized access to those specific accounts. This allows an authenticated attacker to manipulate parameters and initiate unauthorized financial transactions from any user's account to their own

Severity “Critical”

Reproduction Steps

- 1 Log in to the application and capture a legitimate money transfer request using an interception proxy tool.

2 Observe the structure of the POST request body parameters:

from_account=5&to_account=1&amount=1.

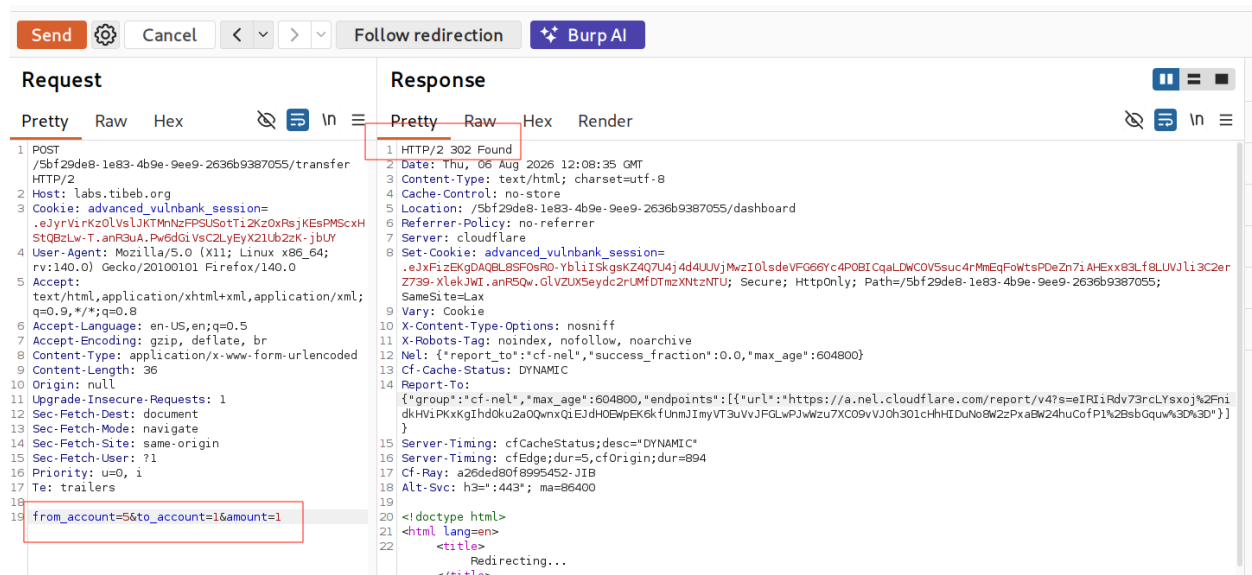
3 Modify the from_account parameter value from your authorized ID (e.g., 5) to a targeted victim's account identifier (e.g., another numerical ID).

4 Keep the to_account parameter set to your own account identifier to receive the funds.

5 Submit the modified request to the server.

6 Observe that the server accepts the transaction, successfully processing the transfer and deducting funds from the victim's account without verification.

Poc



Impact

An attacker can exploit this vulnerability to systematically drain funds from any registered bank account on the platform by enumerating the `from_account` identifier. This bypasses fundamental authorization controls, leading to massive financial fraud, direct theft of customer assets, and a complete breakdown of data isolation between user accounts.

Remediation

Session-Based Authorization: Do not trust the `from_account` parameter provided by the client side. The backend server must dynamically determine the source account identifier directly from the authenticated session cookies or tokens.

Access Control Validation: Implement a strict access control check on the server side to verify that the logged-in user explicitly owns or is authorized to execute transactions on behalf of the requested `from_account` before processing the operation.

Result 5 Business Logic Flaw combined with Improper input Validation

Summary

A Business Logic Flaw combined with Improper Input Validation was identified in the transfer endpoint at `/5bf29de8-1e83-4b9e-9ee9-2636b9387055/transfer`. The backend application fails to perform severe validation checks on transaction logic before processing fund movements. Specifically, the application does not verify if a user has a sufficient balance to cover a transaction, allowing users to transfer money when their account is empty. Furthermore, the endpoint fails to enforce positive-only numeric values on the transaction size, allowing users to input negative values in the amount parameter to manipulate ledger math and arbitrarily credit accounts

Severity “Critical”

Reproduction Steps

- 1 Log in to an account with an empty or low balance and initiate a transfer request.
- 2 Intercept the outbound transaction request using a proxy tool.
- 3 To exploit the insufficient funds logic: Keep the amount parameter higher than the available balance (or perform it when the balance is 0) and submit the request. Observe that the transaction succeeds.
- 4 To exploit the negative value injection logic: Modify the request body payload so that the amount parameter contains a negative integer(e.g., `from_account=5&to_account=1&amount=-5000`).
- 5 Submit the modified payload to the server.
- 6 Observe that the application evaluates the double-negative math ($\text{Balance} - (-5000)$), successfully adding \$5,000 to the source account while bypassing zero-balance restrictions.

Impact

An attacker can exploit this logic flaw to generate infinite arbitrary currency, leading to massive financial fraud and immediate capital extraction from the platform. Failing to restrict negative values and check for sufficient funds destroys the structural integrity of the banking database ledger, allowing users to plunge accounts into unauthorized negative balances or manipulate account values at will without any real financial backing.

Remediation

Sufficient Funds Check: Add a backend conditional check to verify the sender's balance is greater than or equal to the transfer amount before executing.

Positive Value Enforcement: Implement strict input validation to ensure the `amount` parameter is always a positive integer greater than zero.

Database Schema Constraints: Add a `CHECK` constraint at the database layer (e.g., `CHECK (balance >= 0)`) to mechanically reject any transaction that pushes a balance below zero

